

Threads and Multithreading Models

Week 8: Execution contexts, mappings, runtimes, and performance

SDB

Autumn 2026

Intermediate/graduate engineering lecture set

Learning outcomes

By the end of this lecture set, you should be able to:

1. separate process-wide resources from per-thread execution state;
2. distinguish concurrency, parallelism, asynchrony, and preemption;
3. compare M:1, 1:1, and M:N mappings without overgeneralizing blocking;
4. use POSIX creation, joining, detaching, and error handling correctly;
5. explain thread lifecycle, cancellation, signals, and multithreaded `fork()`;
6. reason about pools, work stealing, oversubscription, and Amdahl's law;
7. identify data races, visibility errors, false sharing, and lifetime bugs; and
8. relate native threads to goroutines, virtual threads, fibers, and async tasks.

Guiding question

Which execution abstraction should carry a task, and what state must the OS or runtime preserve to make it safe and scalable?

Road map

- 1 Processes, Threads, and Execution State
- 2 Thread Lifecycle and POSIX Threads
- 3 User and Kernel Thread Mappings
- 4 Pools, Scheduling, and Scalability
- 5 Memory Semantics and Performance Hazards
- 6 Design, Debugging, and Observation
- 7 Synthesis

Road map

- 1 Processes, Threads, and Execution State
- 2 Thread Lifecycle and POSIX Threads
- 3 User and Kernel Thread Mappings
- 4 Pools, Scheduling, and Scalability
- 5 Memory Semantics and Performance Hazards
- 6 Design, Debugging, and Observation
- 7 Synthesis

Process versus thread: protection and execution

Process

A protection and resource container: virtual address space, resource handles, security context, and one or more execution contexts.

Thread

A schedulable execution context with registers, program counter, stack, thread-local state, and scheduling attributes.

Modern correction

In mainstream kernels, threads—not abstract whole processes—are normally dispatched onto CPUs. A single-threaded process simply has one schedulable thread.

What threads share and what they do not

Shared within a process

code and mapped shared libraries
virtual address space, globals, heap
file-descriptor table/open handles
current directory and many credentials
signal dispositions and process-wide limits
synchronization objects in shared memory

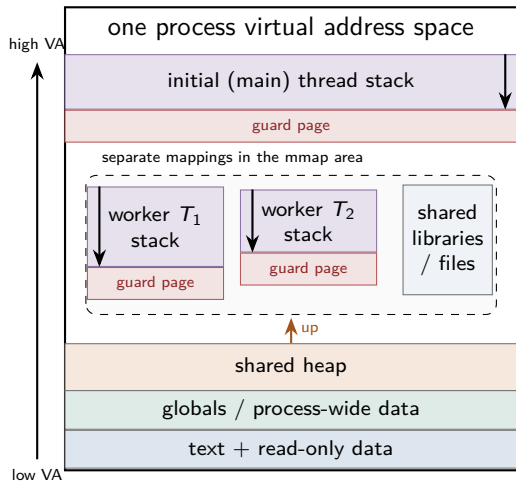
Distinct per thread

program counter and CPU registers
user stack and kernel stack
thread-local storage and `errno` view
signal mask and pending thread signals
scheduler state, priority, affinity
thread identifier and exit value

Consequence

A pointer valid in one thread usually names the same process memory in another. Communication is cheap—and memory corruption is process-wide.

Threaded process memory layout



- No separate “process stack” exists: the initial thread and every worker each own a stack.
- On common downward-growing ABIs, each stack grows toward its lower-address guard page.
- Worker stacks are separate mappings whose placement is implementation-dependent.
- Reserved stack space need not be fully backed by physical memory.
- Exact direction and placement are ABI/OS dependent; this is a common Unix-like convention.

Thread control block: the resumable context

thread ID, state, process/address-space reference
saved registers, program counter, stack pointers
scheduler class, priority, affinity, accounting
signal mask, TLS base, cancellation/runtime metadata
wait object, kernel stack, architecture-specific state

- Exact fields are OS- and architecture-specific.
- Cost may include vector state, TLB effects, cache disruption, and mitigation overhead.

Concurrency is not parallelism

Concurrency

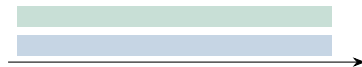
Tasks have overlapping lifetimes and make progress through interleaving, blocking, or asynchronous events.



Asynchrony concerns completion and notification structure; it neither requires nor guarantees another core.

Parallelism

Tasks execute at the same instant on distinct processing resources.



Why threads—and why not always threads?

Goal	Potential benefit	New cost/risk
responsiveness	keep UI/event loop free	coordination and cancellation
overlap CPU and I/O	progress while another task waits	more in-flight state/backpressure
multicore speedup	parallelize independent work	contention and load imbalance
shared state	low-copy communication	races and weak fault isolation
modular servers	one flow per request/task	stack cost and oversubscription
lower creation cost	reuse process resources	no universal fixed speed ratio

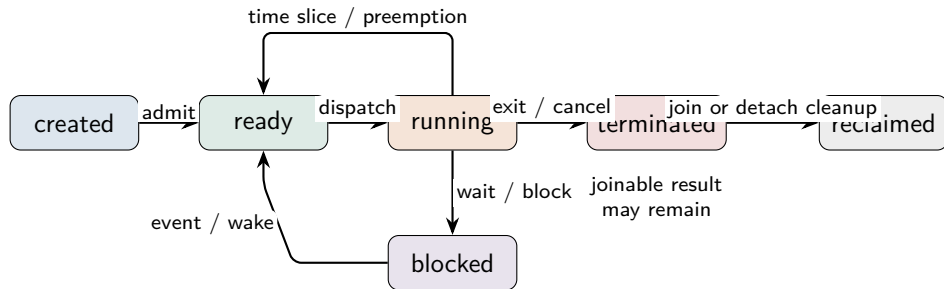
Measure, do not repeat folklore

Creation and switching costs depend on mapping, stack policy, cache state, address-space history, mitigations, and workload. Fixed instruction counts or universal multiplicative claims are not portable facts.

Road map

- 1 Processes, Threads, and Execution State
- 2 Thread Lifecycle and POSIX Threads**
- 3 User and Kernel Thread Mappings
- 4 Pools, Scheduling, and Scalability
- 5 Memory Semantics and Performance Hazards
- 6 Design, Debugging, and Observation
- 7 Synthesis

Thread lifecycle and resource reclamation



- A joinable POSIX thread retains join-related resources until joined.
- A detached thread releases those resources automatically and cannot be joined.

POSIX example I: worker and error contract

```
1 #include <pthread.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <string.h>
5
6 static void check(int rc, const char *what) {
7     if (rc != 0) {
8         fprintf(stderr, "%s: %s\n", what, strerror(rc));
9         exit(EXIT_FAILURE);
10    }
11 }
12
13 static void *worker(void *arg) {
14     printf("worker: %s\n", (const char *)arg);
15     return (void *)42;      /* demonstration-sized payload */
16 }
```

Error contract

Most pthread routines return an error number directly. Report it with `strerror(rc)`; `perror()` reports the unrelated current `errno` unless explicitly assigned.

POSIX example II: create, join, and compile

```
1 int main(void) {  
2     pthread_t tid; void *result;  
3     check(pthread_create(&tid, NULL, worker, "hello"), "pthread_create");  
4     check(pthread_join(tid, &result), "pthread_join");  
5     printf("result = %ld\n", (long)result);  
6     return 0;  
7 }
```

- Compile with `cc -std=c11 -Wall -Wextra -pthread demo.c`.
- The argument string remains valid for the worker's lifetime.
- Joining both waits for completion and reclaims join-related resources.

Join, detach, and ownership

Joinable thread

- another thread calls `pthread_join`;
- join waits for completion and obtains the result;
- exactly one successful joiner is expected.

Detached thread

- created detached or later detached;
- resources reclaimed at exit;
- no later join or result collection.

Design rule

Every created thread needs a lifetime owner: join it, detach it, or place it under a structured runtime that guarantees cleanup.

Argument and result lifetime hazards

Unsafe pattern

Pass the address of a loop variable or stack object, then change it or leave its scope before the worker reads it.

Safe patterns

- stable per-task objects;
- an array alive until all joins;
- copied immutable values;
- documented result ownership.

Subtlety

Shared addressability does not imply valid lifetime, synchronized access, or ownership. These are separate obligations.

Cancellation should be cooperative

- Abrupt asynchronous cancellation can leave locks held and invariants broken.
- POSIX deferred cancellation acts at cancellation points; cleanup handlers can release resources.
- Other runtimes use interruption flags, cancellation tokens, deadlines, or closed channels.
- Blocking APIs must participate or be interruptible for timely shutdown.

Structured cancellation

Parent scopes should know child tasks, propagate cancellation, and await cleanup before the scope exits.

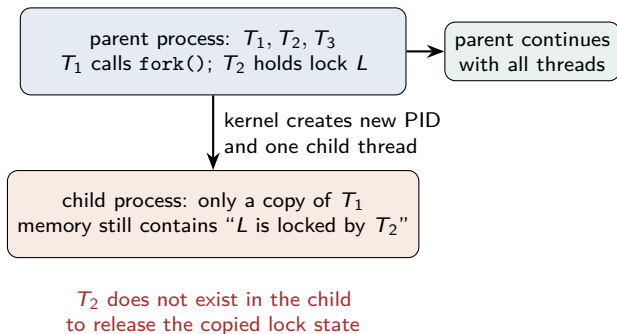
Signals in a multithreaded process

- Signal dispositions are generally process-wide; signal masks are per thread.
- A process-directed signal may be delivered to one eligible unblocked thread.
- A thread-directed signal targets a particular thread.
- Synchronous faults arise in the faulting thread, but the default action can terminate the process.

Robust pattern

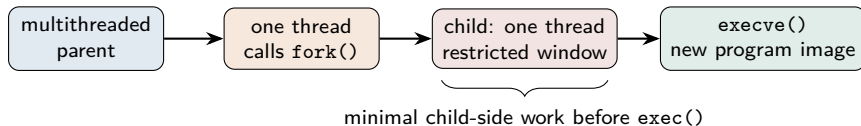
Block selected asynchronous signals in workers and dedicate one thread to synchronous receipt with `sigwait()`-style APIs.

Multithreaded `fork()`: what the system creates



- The child receives a new process identity and one execution context: the caller's.
- Virtual-memory contents are logically copied, normally using copy-on-write page mappings.
- Open descriptors are duplicated and still refer to the same underlying open-file descriptions.
- Mutex words and library state are copied as ordinary memory; ownership is not reconstructed.

The child-side fork-exec window



Preferred paths

- promptly invoke the new program with `exec`;
- call `_exit()` on child-side failure;
- prefer `posix_spawn()` when it expresses the setup.

Role of `pthread_atfork()`

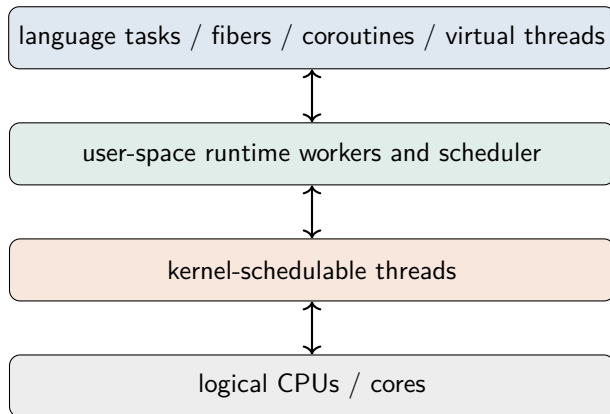
Prepare, parent, and child handlers can coordinate application-owned locks. They cannot reliably repair hidden locks in arbitrary libraries.

Between `fork()` and `exec()` in the multithreaded child, avoid allocation, buffered I/O, and library code that may acquire copied locks; use only operations permitted in this context.

Road map

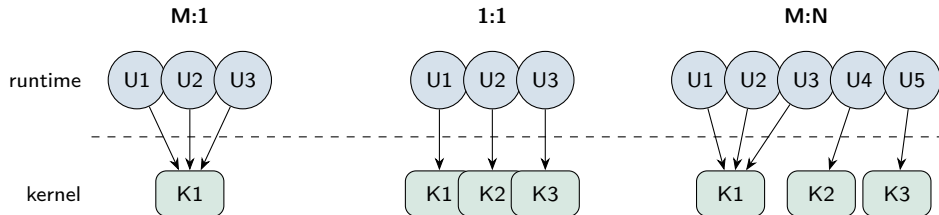
- 1 Processes, Threads, and Execution State
- 2 Thread Lifecycle and POSIX Threads
- 3 User and Kernel Thread Mappings**
- 4 Pools, Scheduling, and Scalability
- 5 Memory Semantics and Performance Hazards
- 6 Design, Debugging, and Observation
- 7 Synthesis

Separate three software layers



- User thread may mean an API thread, a runtime task, or a user-managed stack.
- The mapping determines visibility, blocking behavior, parallelism, and control.

M:1, 1:1, and M:N at a glance



illustrative instant; runtime assignments can change

	M:1	1:1	M:N
kernel visibility	one carrier	every API thread	carrier subset
multicore parallelism	at most one	direct	up to N carriers
runtime complexity	low/moderate	low mapping overhead	high

Blocking depends on mapping and integration

Event	M:1	1:1	M:N runtime
kernel blocks carrier	all user tasks stop	caller stops	other carriers may run
runtime-aware async I/O	another task can run	another thread runs	carrier can be reused
page fault	sole carrier blocks	faulting thread blocks	other carriers progress
foreign blocking call	blocks unless adapted	blocks caller	may lose/replace carrier

Correction to the textbook slogan

A blocking call stops the whole process only for a pure M:1 runtime using an unadapted blocking operation, not for every user-managed threading system.

One-to-one: direct OS integration

- Each application thread corresponds to a kernel-schedulable thread.
- Blocking, priorities, affinity, signals, and profiling integrate directly with the OS.
- Different threads from one process can execute simultaneously.
- Each consumes kernel metadata and stack/address-space resources; enormous counts are costly.

Common deployment

Native POSIX threads on Linux and platform threads on mainstream desktop/server systems typically expose 1:1 behavior.

Many-to-many: runtime scheduling over carriers

- Many logical tasks are multiplexed over a bounded or elastic carrier set.
- Runtime can use lightweight stacks, task-aware blocking, local queues, and work stealing.
- Parallelism is bounded by carriers and CPU capacity, not logical task count.
- Native calls, TLS assumptions, pinning, debugging, and profiling add complexity.

Logical task count is not CPU speed

A million lightweight tasks may improve throughput for blocked workloads, but cannot make fixed CPU work exceed core capacity or algorithmic parallelism.

Modern examples: goroutines and virtual threads

- Go multiplexes goroutines over OS threads, and its runtime can preempt goroutines; programs must not assume a schedule order.
- Java virtual threads are user-mode `Thread` instances scheduled over carrier platform threads.
- Virtual threads target high-throughput workloads with abundant blocking, especially I/O; they are not faster CPU threads.
- Both retain a thread-like blocking style while moving multiplexing into the language runtime.

Abstraction benefit

Runtime-aware tasks can make blocking source code scale like event-driven code when blocking can release or unmount the carrier.

Source basis: Go documentation and OpenJDK JEP 444 / Java virtual-thread documentation, accessed August 2026.

Stackful threads versus stackless async tasks

	Stackful thread/fiber	Stackless async state machine
suspension	almost anywhere runtime permits	explicit <code>await</code> / <code>yield</code> points
state	preserved call stack	compiler-generated state object
coding style	direct sequential flow	async propagation/function coloring
memory	stack or segmented stack	task object and captured state
migration	runtime-dependent	executor-dependent
debugging	familiar stack per task	logical async stacks needed

Coroutines describe suspend/resume control flow; M:1 versus M:N depends on the executor and runtime.

Road map

- 1 Processes, Threads, and Execution State
- 2 Thread Lifecycle and POSIX Threads
- 3 User and Kernel Thread Mappings
- 4 Pools, Scheduling, and Scalability**
- 5 Memory Semantics and Performance Hazards
- 6 Design, Debugging, and Observation
- 7 Synthesis

Thread-per-task versus a worker pool

Thread per task

- simple ownership and blocking style;
- natural stack isolation;
- native threads cost resources at high concurrency.

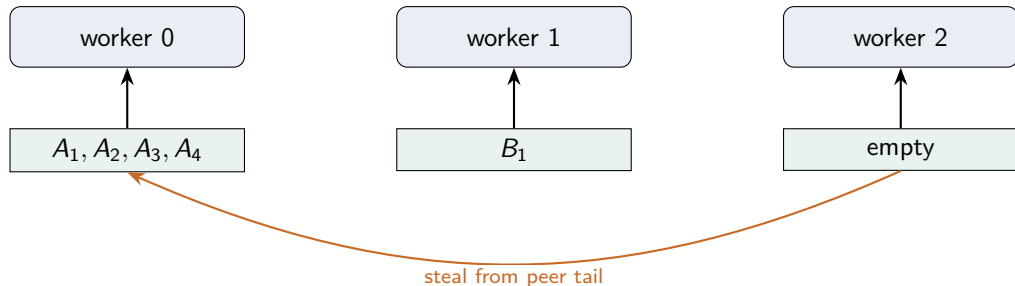
Pool of workers

- amortizes creation and bounds workers;
- requires queues and backpressure;
- blocking tasks can exhaust the pool.

Queueing is part of the API contract

An unbounded queue converts overload into latency and memory growth. A bounded queue requires rejection, shedding, or producer backpressure.

Work stealing balances irregular parallelism



- Workers prefer local queues for locality and low contention.
- Idle workers steal tasks, improving balance for recursive or irregular work.
- Blocking, priorities, affinity, and non-stealable tasks complicate the ideal.

Oversubscription: runnable threads exceed CPUs

If R runnable CPU-bound threads compete for P logical CPUs:

- $R \leq P$ can underutilize CPUs if tasks block or are imbalanced;
- $R \gg P$ increases preemption, cache/TLB churn, contention, and tail latency;
- modest oversubscription can hide blocking but must be workload-driven;
- simultaneous multithreading does not make each logical CPU a full core.

Sizing principle

Tie CPU-bound pools to measured CPU capacity. Give blocking pools explicit concurrency and backpressure limits instead of arbitrary huge sizes.

Amdahl's law bounds ideal speedup

If fraction p is perfectly parallel and $1 - p$ remains serial,

$$S(N) = \frac{1}{(1 - p) + p/N}.$$

For $p = 0.95$ and $N = 16$:

$$S(16) = \frac{1}{0.05 + 0.95/16} \approx 9.14,$$

not $16\times$.

- Synchronization, communication, imbalance, and memory bandwidth reduce speedup further.
- Increasing problem size can change p ; weak scaling asks a different question.

Little's law connects concurrency and throughput

For a stable system,

$$L = \lambda W,$$

where L is average in-flight work, λ throughput, and W average response time.

Example

At $\lambda = 2,000$ requests/s and $W = 50$ ms, average concurrency is $L = 2000 \times 0.05 = 100$ requests.

Capacity lesson

More threads do not create capacity. If arrivals exceed service capacity, queues and latency grow; bound concurrency and apply backpressure.

Road map

- 1 Processes, Threads, and Execution State
- 2 Thread Lifecycle and POSIX Threads
- 3 User and Kernel Thread Mappings
- 4 Pools, Scheduling, and Scalability
- 5 Memory Semantics and Performance Hazards**
- 6 Design, Debugging, and Observation
- 7 Synthesis

A data race is more than unlucky interleaving

Conflicting accesses race when concurrent, at least one is a write, and no synchronization orders them.

Naive increment

`x++` is read-modify-write. Two threads can read one old value and lose an update.

Language consequence

In C/C++, a race on a non-atomic object is undefined behavior, allowing compiler transformations beyond a lost update.

Correctness requires atomicity and a defined memory-order relationship for visibility.

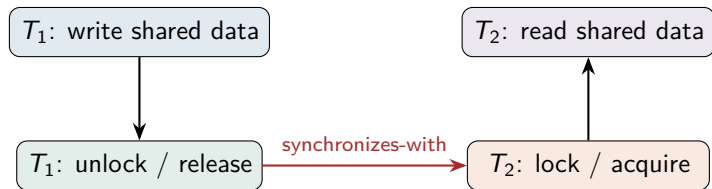
Atomic counter versus protected invariant

```
1 #include <stdatomic.h>
2
3 static _Atomic unsigned long requests = 0;
4
5 void count_request(void) {
6     atomic_fetch_add_explicit(&requests, 1, memory_order_relaxed);
7 }
```

- Relaxed atomicity suits an independent statistics counter when it publishes no other data.
- A mutex is clearer for multi-field invariants such as balance plus transaction log.
- Lock-free does not imply faster, fair, wait-free, or free of cache contention.

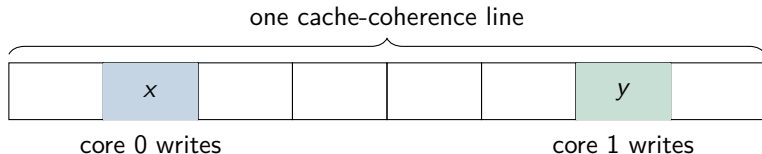
Happens-before: the visibility contract

Synchronization creates ordering edges constraining compiler and hardware reordering.



By transitivity, the write happens-before the read, so the synchronized reader observes a state allowed by the language memory model.

False sharing: independent data, shared cache line



- x and y need not be logically shared, yet writes invalidate the full line.
- Symptoms include poor scaling and coherence traffic without a lock hotspot.
- Padding/alignment can help but changes layout and wastes cache capacity.

Thread-local storage is isolation, not synchronization

- TLS gives each thread a distinct instance of a named object.
- Useful for per-thread caches, random state, error state, and scratch buffers.
- TLS reduces sharing but can increase footprint and complicate aggregation.
- Logical tasks multiplexed over workers must not confuse carrier TLS with task-local state.

Runtime distinction

Thread-local, fiber-local, task-local, and request-context storage have different lifetimes and migration behavior.

Fault isolation and failure containment

	Separate processes	Threads in one process
address isolation	hardware-enforced by default	shared; pointer bugs cross components
communication	IPC/serialization/shared maps	direct loads/stores
crash scope	often one process/service	often whole process
resource control	process/cgroup controls	shared limits, per-thread CPU policy
deployment	heavier, independently restartable	lightweight composition

Architecture principle

Choose process boundaries for trust and failure containment; choose threads/tasks for efficient cooperation inside a trust boundary.

Road map

- 1 Processes, Threads, and Execution State
- 2 Thread Lifecycle and POSIX Threads
- 3 User and Kernel Thread Mappings
- 4 Pools, Scheduling, and Scalability
- 5 Memory Semantics and Performance Hazards
- 6 Design, Debugging, and Observation**
- 7 Synthesis

Choosing an execution model

Workload	Good starting point	Primary concern
CPU-parallel numerical kernel many blocked network requests GUI application untrusted plugins/services recursive irregular tasks real-time control	bounded native worker pool async or lightweight M:N tasks event thread + bounded workers process isolation + IPC work-stealing pool bounded native threads	locality and bandwidth backpressure and pinning never block UI dispatcher containment and permissions granularity and stealing cost WCET, locks, priorities

Debugging nondeterministic failures

- Reproduce under varied CPU counts, affinity, timing, and optimization.
- Use race detectors/sanitizers for unsynchronized memory access.
- Record lock order, blocking duration, queue length, and ownership.
- Prefer deterministic tests for concurrent components where possible.
- Capture thread dumps and scheduler traces during stalls.
- Distinguish deadlock, livelock, starvation, inversion, and overload.

Heisenbug warning

Logging and debugging alter timing and layout. No failure under a debugger is not evidence of correctness.

Observing threads on Linux

```
1 # List kernel-visible threads in one process
2 ps -L -p "$PID" -o pid,tid,psr,stat,pri,ni,pcpu,comm
3
4 # Per-thread procfs entries and interactive view
5 ls /proc/$PID/task
6 cat /proc/$PID/task/$TID/status
7 top -H -p "$PID"
8
9 # Trace creation, waiting, and termination
10 strace -f -e clone,clone3,futex,exit ./program
```

An M:N runtime may expose only carrier OS threads here; use runtime-specific diagnostics for logical tasks.

Code-review checklist for thread creation

1. Who owns the task and how is it stopped?
2. Is its argument valid until consumed?
3. Is every joinable thread joined exactly once, or detached?
4. What bounds active and queued work?
5. What synchronization establishes happens-before?
6. Can blocking exhaust a pool or pin a carrier?
7. What happens on partial startup failure?
8. Are errors, cancellation, and results propagated?

Road map

- 1 Processes, Threads, and Execution State
- 2 Thread Lifecycle and POSIX Threads
- 3 User and Kernel Thread Mappings
- 4 Pools, Scheduling, and Scalability
- 5 Memory Semantics and Performance Hazards
- 6 Design, Debugging, and Observation
- 7 **Synthesis**

Misconceptions corrected

Misconception	Precise statement
Process is always the CPU scheduling unit.	⇒ Mainstream kernels normally dispatch threads/tasks.
User threads cannot run in parallel.	⇒ M:1 cannot; M:N can use multiple carriers.
Any blocking call blocks every user thread.	⇒ It depends on mapping and runtime-aware I/O.
Threads are always much faster than processes.	⇒ Cost depends on implementation and workload state.
volatile fixes races.	⇒ Use language-defined atomics or synchronization.
More threads produce more CPU throughput.	⇒ Cores, serial work, and bottlenecks bound speedup.
Virtual threads make CPU code faster.	⇒ They primarily scale blocked concurrency.
Different variables cannot contend.	⇒ False sharing couples writes in one cache line.

Key takeaways

- A process is a protection/resource container; a thread is an execution context.
- Shared addressing lowers communication cost and fault isolation.
- M:1, 1:1, and M:N determine visibility, blocking, and parallelism.
- Correct lifecycle management needs explicit ownership, join/detach, and cancellation.
- Pools and lightweight tasks require bounded queues and backpressure.
- Amdahl's law, balance, and memory bandwidth limit speedup.
- Synchronization defines atomicity and visibility; false sharing can dominate cost.
- Runtime multiplexing cannot evade finite CPU capacity.

Next week: synchronization and memory ordering

- critical-section safety and liveness requirements;
- mutexes, semaphores, condition variables, and monitors;
- atomic read–modify–write instructions;
- acquire/release and sequential consistency;
- deadlock conditions, avoidance, and lock ordering;
- bounded-buffer and readers–writers problems;
- lock-free versus wait-free progress.

Preparation

Review invariants, interleavings, cache coherence, and the difference between atomicity and visibility.

Road map

- 8 Appendix: Code and Exploration
 - Code examples
 - Exploratory topics
 - Debugging exercises: deliberate faults

Knowledge check: execution state

1. Which resources are process-wide and which are per-thread?
2. Why does each thread require a separate stack?
3. Can a single-threaded process run on two cores simultaneously?
4. Distinguish concurrency, parallelism, and asynchrony.
5. Why can one thread's invalid pointer terminate all threads?
6. How can TLS fail for tasks that migrate carriers?

Exercise: repair a pthread program

A loop creates eight threads and passes `&i` as each argument. The loop ends before joins, failures call `perror()`, and some successful threads are never joined.

1. Identify the lifetime and data-race problem.
2. Replace it with stable per-thread argument storage.
3. Correct error reporting using returned error codes.
4. Handle partial creation failure without leaks.
5. Define result ownership and cleanup.
6. Compile with warnings and run a thread sanitizer.

Exercise: choose a mapping and pool size

Compare three services:

1. 32 CPU-heavy image transforms on a 16-core server;
2. 100,000 mostly blocked network connections;
3. 20 untrusted plugins processing customer documents.

For each:

- choose processes, 1:1 threads, M:N tasks, or an event loop;
- bound active and queued work;
- identify the bottleneck and failure boundary;
- state what evidence would change the design.

Exercise: Amdahl and overhead

A program is 92% parallel. Each parallel worker adds 0.3% of original runtime in scheduling and communication overhead.

1. Compute ideal speedup for $N = 2, 4, 8, 16, 32$.
2. Add overhead $0.003N$ to the normalized denominator.
3. Which N gives the best modeled speedup?
4. Why might memory bandwidth make performance peak earlier?
5. Distinguish strong scaling from weak scaling.

Exercise: diagnose a scalability collapse

A counter benchmark scales to four cores, then slows sharply. No mutex appears; each worker updates a distinct array element.

1. Draw a plausible false-sharing layout.
2. What profiling evidence would support the diagnosis?
3. Compare padding, sharding, and batched aggregation.
4. Explain padding's memory-footprint trade-off.
5. Why might relaxed atomics not fix performance?

Code example: parallel sum worker

```
1 #include <pthread.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <string.h>
5 enum { N = 4 };
6 struct job { const int *x; size_t lo, hi; long sum; };
7 static void *sum_part(void *p) {
8     struct job *j = p;
9     j->sum = 0;
10    for (size_t i = j->lo; i < j->hi; ++i) j->sum += j->x[i];
11    return NULL;
12 }
```

- Each worker reads the shared input and writes only its own job record.
- The driver shown next supplies non-overlapping ranges and joins before reducing results.

Combine with the next slide; compile using `cc -std=c11 -Wall -Wextra -pthread sum.c`. Expected output: `sum = 78`.

Code example: create, join, and reduce

```
1 int main(void) {
2     static const int x[12] = {1,2,3,4,5,6,7,8,9,10,11,12};
3     pthread_t t[N]; struct job j[N]; size_t made = 0;
4     for (size_t i = 0; i < N; ++i) {
5         j[i] = (struct job){x, 3*i, 3*(i+1), 0};
6         int e = pthread_create(&t[i], NULL, sum_part, &j[i]);
7         if (e) {
8             fprintf(stderr, "create: %s\n", strerror(e));
9             while (made) pthread_join(t[--made], NULL);
10            return EXIT_FAILURE;
11        }
12        ++made;
13    }
14    long total = 0;
15    for (size_t i = 0; i < N; ++i) {
16        int e = pthread_join(t[i], NULL);
17        if (e) { fprintf(stderr, "join: %s\n", strerror(e)); return EXIT_FAILURE; }
18        total += j[i].sum;
19    }
20    printf("sum = %ld\n", total);
21    return EXIT_SUCCESS;
22 }
```

Code example: constrained fork-exec-wait

```
1 #include <errno.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <sys/wait.h>
5 #include <unistd.h>
6 extern char **environ;
7 int main(void) {
8     pid_t child = fork();
9     if (child < 0) { perror("fork"); return EXIT_FAILURE; }
10    if (child == 0) {
11        char *const argv[] = {"/usr/bin/printf", "child ran\n", NULL};
12        execve(argv[0], argv, environ);
13        _exit(127);
14    }
15    int status;
16    pid_t r;
17    do { r = waitpid(child, &status, 0); } while (r == -1 && errno == EINTR);
18    if (r == -1) { perror("waitpid"); return EXIT_FAILURE; }
19    if (WIFEXITED(status)) printf("exit: %d\n", WEXITSTATUS(status));
20    return EXIT_SUCCESS;
21 }
```

Exploration: runtimes and execution semantics

- **POSIX lifecycle:** attributes, cancellation points, signal masks, cleanup handlers, and at-fork handlers;
- **Memory models:** compare C/C++ atomics and Java happens-before guarantees;
- **Task runtimes:** work stealing, fork-join decomposition, structured concurrency, and task-local state;
- **Go:** inspect goroutine scheduling, blocking system calls, and runtime trace output;
- **Java:** examine virtual-thread carrier scheduling, pinning conditions, and observability tools;
- **Rust:** compare async executors, task migration, ownership, and Send constraints.

Mini-investigation

Choose one runtime. Trace a blocking operation and explain whether it blocks a kernel thread, a carrier, a logical task, or the whole process.

Exploration: scalability, locality, and testing

- **Memory hierarchy:** measure false sharing, cache coherence traffic, and NUMA placement effects;
- **Scheduling:** compare work stealing with static partitioning under balanced and skewed workloads;
- **Correctness tools:** compare race detectors, thread sanitizers, and model checking;
- **Reproducibility:** design deterministic tests for cancellation, shutdown, and partial startup failure;
- **Fault boundaries:** compare thread pools with worker processes for untrusted or crash-prone tasks;
- **Performance study:** report throughput and latency percentiles while sweeping worker count and queue capacity.

Suggested project

Build a bounded task executor. Document its ownership model, queue policy, shutdown behavior, and measurements under CPU-heavy, blocking, and imbalanced workloads.

Exercise: a shared counter that loses updates

```
1 volatile long total = 0;
2 static void *worker(void *unused) {
3     (void)unused;
4     for (int i = 0; i < 100000; ++i)
5         ++total;
6     return NULL;
7 }
```

1. Start two workers and join both. Is the final value guaranteed to be 200000?
2. Identify the language-level fault and explain why `volatile` does not establish thread synchronization.
3. Predict how a race detector will characterize this access.

Hint

Expand `++total` into its conceptual read, modify, and write steps; consider two workers interleaving those steps.

Exercise: thread arguments with the wrong lifetime

```
1 static void *print_id(void *p) {  
2     int id = *(int *)p;  
3     printf("worker %d\\n", id);  
4     return NULL;  
5 }  
6 int main(void) {  
7     pthread_t t[8];  
8     for (int i = 0; i < 8; ++i)  
9         pthread_create(&t[i], NULL, print_id, &i);  
10    for (int i = 0; i < 8; ++i) pthread_join(t[i], NULL);  
11 }
```

1. List the different outputs that might appear, and explain why output order is not the only problem.
2. Identify the lifetime and concurrent-access hazards.
3. Propose a test or instrumentation that could make the bug easier to observe.

Hint

Track exactly which object each worker receives, how long that object lives, and whether the loop can change it before a worker reads it.

Exercise: inconsistent lock ordering

```
1 pthread_mutex_t a = PTHREAD_MUTEX_INITIALIZER;
2 pthread_mutex_t b = PTHREAD_MUTEX_INITIALIZER;
3 /* Worker A */
4 pthread_mutex_lock(&a); pthread_mutex_lock(&b);
5 /* critical section */
6 pthread_mutex_unlock(&b); pthread_mutex_unlock(&a);
7 /* Worker B performs the same work with reverse acquisition order. */
8 pthread_mutex_lock(&b); pthread_mutex_lock(&a);
9 /* critical section */
10 pthread_mutex_unlock(&a); pthread_mutex_unlock(&b);
```

1. Give an interleaving in which both workers stop making progress.
2. Draw the wait-for/resource-allocation graph and identify the cycle.
3. Describe a design invariant that would prevent this class of failure; do not rely on timing.

Hint

Assume each worker acquires its first mutex before the other worker attempts its second. Who owns each lock, and what is each waiting for?

Exercise: condition-variable predicate bug

```
1 /* mutex is held on entry; count is the number of queued items */
2 static void consume_one(void) {
3     if (count == 0)
4         pthread_cond_wait(&ready, &mutex);
5     --count;                /* claim one item */
6     pthread_mutex_unlock(&mutex);
7 }
8 /* Producer adds one item while holding mutex, then: */
9 pthread_cond_broadcast(&ready);
```

1. With two consumers waiting and one item added, describe a legal execution that breaks the queue invariant.
2. Explain what the condition-variable notification does—and what it does not promise.
3. Identify another wakeup case the consumer must be prepared to handle.

Hint

Treat the shared predicate as the truth; a wakeup is only a reason to re-check it. Consider both competing consumers and spurious wakeups.